In [ ]:
```
from google.colab import files
files.upload()
```

In [ ]:
```
import pandas as pd

try:
    df = pd.read_csv('/content/Global_Superstore2 .csv', encoding='latin1')
except UnicodeDecodeError:
    df = pd.read_csv('/content/Global_Superstore2 .csv', encoding='ISO-8859-1')

df.head()
data = df.copy()
data
```

Out[ ]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	
0	32298	CA-2012-124891	31-07-2012	31-07-2012	Same Day	RH-19495	Rick Hansen	Consumer	New
1	26341	IN-2013-77878	05-02-2013	07-02-2013	Second Class	JR-16210	Justin Ritter	Corporate	Wollon
2	25330	IN-2013-71249	17-10-2013	18-10-2013	First Class	CR-12730	Craig Reiter	Consumer	Bris
3	13524	ES-2013-1579342	28-01-2013	30-01-2013	First Class	KM-16375	Katherine Murray	Home Office	
4	47221	SG-2013-4320	05-11-2013	06-11-2013	Same Day	RH-9495	Rick Hansen	Consumer	
...	...	...	...	...	...	...	...	...	
51285	29002	IN-2014-62366	19-06-2014	19-06-2014	Same Day	KE-16420	Katrina Edelman	Corporate	
51286	35398	US-2014-102288	20-06-2014	24-06-2014	Standard Class	ZC-21910	Zuschuss Carroll	Consumer	Ho
51287	40470	US-2013-155768	02-12-2013	02-12-2013	Same Day	LB-16795	Laurel Beltran	Home Office	O
51288	9596	MX-2012-140767	18-02-2012	22-02-2012	Standard Class	RB-19795	Ross Baird	Home Office	Val
51289	6147	MX-2012-134460	22-05-2012	26-05-2012	Second Class	MC-18100	Mick Crebagga	Consumer	Tip

51290 rows × 24 columns

In [ ]:

```python
import pandas as pd
import numpy as np

# 🔷 Clean column names (avoid hidden space issues)
data.columns = data.columns.str.strip()

# 🔷 Ensure correct data type
data['Shipping Cost'] = pd.to_numeric(data['Shipping Cost'], errors='coerce')

# 🔷 Group and aggregate
city_shipping = data.groupby('City', as_index=False)['Shipping Cost'].agg(
    Total_Shipping_Cost='sum',
    Average_Shipping_Cost='mean'
)

# 🔷 Get top and bottom cities (Total Shipping Cost)
top_city_total = city_shipping.loc[city_shipping['Total_Shipping_Cost'].idxmax()
bottom_city_total = city_shipping.loc[city_shipping['Total_Shipping_Cost'].idxmi

# 🔷 Get top and bottom cities (Average Shipping Cost)
top_city_avg = city_shipping.loc[city_shipping['Average_Shipping_Cost'].idxmax()
bottom_city_avg = city_shipping.loc[city_shipping['Average_Shipping_Cost'].idxmi

# 🔷 Overall average
average_city_shipping_cost = city_shipping['Average_Shipping_Cost'].mean()

# 🔷 Create summary table
summary_df = pd.DataFrame({
    'Metric': [
        'Top City by Total Shipping Cost',
        'Bottom City by Total Shipping Cost',
        'Top City by Average Shipping Cost',
        'Bottom City by Average Shipping Cost',
        'Average Shipping Cost Across All Cities'
    ],
    'City': [
        top_city_total['City'],
        bottom_city_total['City'],
        top_city_avg['City'],
        bottom_city_avg['City'],
        np.nan
    ],
    'Total Shipping Cost': [
        top_city_total['Total_Shipping_Cost'],
        bottom_city_total['Total_Shipping_Cost'],
        np.nan,
        np.nan,
        np.nan
    ],
    'Average Shipping Cost': [
        np.nan,
        np.nan,
        top_city_avg['Average_Shipping_Cost'],
        bottom_city_avg['Average_Shipping_Cost'],
        average_city_shipping_cost
    ]
})
```

```
# 🔷 Display result
print(summary_df)
```

```
                                  Metric            City  \
0          Top City by Total Shipping Cost   New York City
1       Bottom City by Total Shipping Cost  Victoria Falls
2        Top City by Average Shipping Cost          Kamina
3     Bottom City by Average Shipping Cost  Victoria Falls
4  Average Shipping Cost Across All Cities             NaN

   Total Shipping Cost  Average Shipping Cost
0             26948.17                    NaN
1                 0.09                    NaN
2                  NaN             678.150000
3                  NaN               0.090000
4                  NaN              27.656888
```

In [ ]:
```
# calculate totaland average shipping cost by region
region_shipping = data.groupby('Region')['Shipping Cost'].agg(['sum', 'mean']).r
region_shipping.columns = ['Region', 'Total Shipping Cost', 'Average Shipping Co
```

In [ ]:
```
# Get top and bottom regions by total shipping cost
top_region_total = region_shipping.loc[region_shipping['Total Shipping Cost'].id
```

In [ ]:
```
# Get top and bottom regions by average shipping cost
top_region_total = region_shipping.loc[region_shipping['Average Shipping Cost'].
bottom_region_avg = region_shipping.loc[region_shipping['Average Shipping Cost']
```

In [ ]:
```
# Calculate average shipping cost across all regions
average_region_shipping_cost = region_shipping['Average Shipping Cost'].mean()
```

In [ ]:
```
print(data.columns)
```

```
Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',
       'Customer ID', 'Customer Name', 'Segment', 'City', 'State', 'Country',
       'Postal Code', 'Market', 'Region', 'Product ID', 'Category',
       'Sub-Category', 'Product Name', 'Sales', 'Quantity', 'Discount',
       'Profit', 'Shipping Cost', 'Order Priority'],
      dtype='object')
```

In [ ]:
```
total_shipping = data.groupby('Region')['Shipping Cost'].sum().reset_index()
total_shipping.rename(columns={'Shipping Cost': 'Total Shipping Cost'}, inplace=
top_region_total = total_shipping.loc[total_shipping['Total Shipping Cost'].idxm
bottom_region_total = total_shipping.loc[total_shipping['Total Shipping Cost'].i
avg_shipping = data.groupby('Region')['Shipping Cost'].mean().reset_index()
avg_shipping.rename(columns={'Shipping Cost': 'Average Shipping Cost'}, inplace=

top_region_avg = avg_shipping.loc[avg_shipping['Average Shipping Cost'].idxmax()
bottom_region_avg = avg_shipping.loc[avg_shipping['Average Shipping Cost'].idxmi
```

In [ ]:
```
summary_region_df = pd.DataFrame({
    'metric': [
        'Top Region by Total Shipping Cost',
        'Bottom Region by Total Shipping Cost',
        'Top Region by Average Shipping Cost',
        'Bottom Region by Average Shipping Cost',
        'Average Shipping Cost Across All Regions'
    ],
    'Region': [
```

```
        top_region_total['Region'],
        bottom_region_total['Region'],
        top_region_avg['Region'],
        bottom_region_avg['Region'],
        'N/A'
    ],
    'Total Shipping Cost': [
        top_region_total['Total Shipping Cost'],
        bottom_region_total['Total Shipping Cost'],
        None,
        None,
        data['Shipping Cost'].mean()
    ]
})
```

In [ ]:

```
import numpy as np

import pandas as pd
import numpy as np

#  Clean column names
data.columns = data.columns.str.strip()

#  Ensure numeric type
data['Shipping Cost'] = pd.to_numeric(data['Shipping Cost'], errors='coerce')

#  Group by Region
region_shipping = data.groupby('Region', as_index=False)['Shipping Cost'].agg(
    Total_Shipping_Cost='sum',
    Average_Shipping_Cost='mean'
)

#  Top & Bottom Regions (Total Shipping Cost)
top_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost'].id
bottom_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost']

#  Top & Bottom Regions (Average Shipping Cost)
top_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost'].id
bottom_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost']

#  Overall average
average_region_shipping_cost = region_shipping['Average_Shipping_Cost'].mean()

#  Create summary DataFrame
summary_region_df = pd.DataFrame({
    'Metric': [
        'Top Region by Total Shipping Cost',
        'Bottom Region by Total Shipping Cost',
        'Top Region by Average Shipping Cost',
        'Bottom Region by Average Shipping Cost',
        'Average Shipping Cost Across All Regions'
    ],
    'Region': [
        top_region_total['Region'],
        bottom_region_total['Region'],
        top_region_avg['Region'],
        bottom_region_avg['Region'],
        np.nan
    ],
    'Total_Shipping_Cost': [
```

```
        top_region_total['Total_Shipping_Cost'],
        bottom_region_total['Total_Shipping_Cost'],
        np.nan,
        np.nan,
        np.nan
    ],
    'Average_Shipping_Cost': [
        np.nan,
        np.nan,
        top_region_avg['Average_Shipping_Cost'],
        bottom_region_avg['Average_Shipping_Cost'],
        average_region_shipping_cost
    ]
})

# 🔷 Display result
print(summary_region_df)
```

```
                                    Metric      Region  Total_Shipping_Cost  \
0         Top Region by Total Shipping Cost     Central            296940.56
1      Bottom Region by Total Shipping Cost      Canada              7405.63
2       Top Region by Average Shipping Cost  North Asia                  NaN
3    Bottom Region by Average Shipping Cost        EMEA                  NaN
4  Average Shipping Cost Across All Regions         NaN                  NaN

   Average_Shipping_Cost
0                    NaN
1                    NaN
2              40.646112
3              17.573221
4              26.867990
```

In [ ]:
```
# Group by Region
region_shipping = data.groupby('Region', as_index=False)['Shipping Cost'].agg(
    Total_Shipping_Cost='sum',
    Average_Shipping_Cost='mean'
)

#  Top & Bottom (Total Shipping Cost)
top_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost'].id
bottom_region_total = region_shipping.loc[region_shipping['Total_Shipping_Cost']

#  Top & Bottom (Average Shipping Cost)
top_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost'].id
bottom_region_avg = region_shipping.loc[region_shipping['Average_Shipping_Cost']

#  Overall average
average_region_shipping_cost = region_shipping['Average_Shipping_Cost'].mean()
```

In [ ]:
```
# Calculate total and average shipping cost by region
region_shipping = data.groupby('Region')['Shipping Cost'].agg(['sum','mean']).re
region_shipping.columns = ['Region', 'Total Shipping Cost', 'Average Shipping Co
```

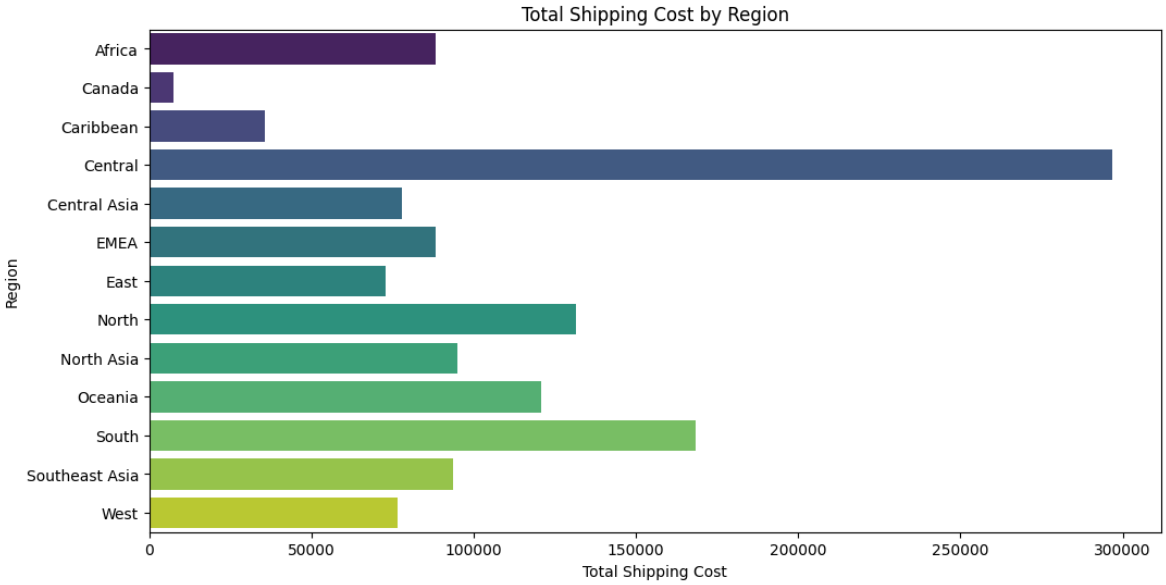
In [ ]:
```
import seaborn as sns
import matplotlib.pyplot as plt
# Plot total shipping cost by region
plt.figure(figsize=(12, 6))
sns.barplot(x='Total Shipping Cost', y='Region', data=region_shipping, palette='
plt.title('Total Shipping Cost by Region')
plt.xlabel('Total Shipping Cost')
```

```
plt.ylabel('Region')
plt.show()
```

```
/tmp/ipykernel_886/404086954.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Total Shipping Cost', y='Region', data=region_shipping, palette
='viridis')
```



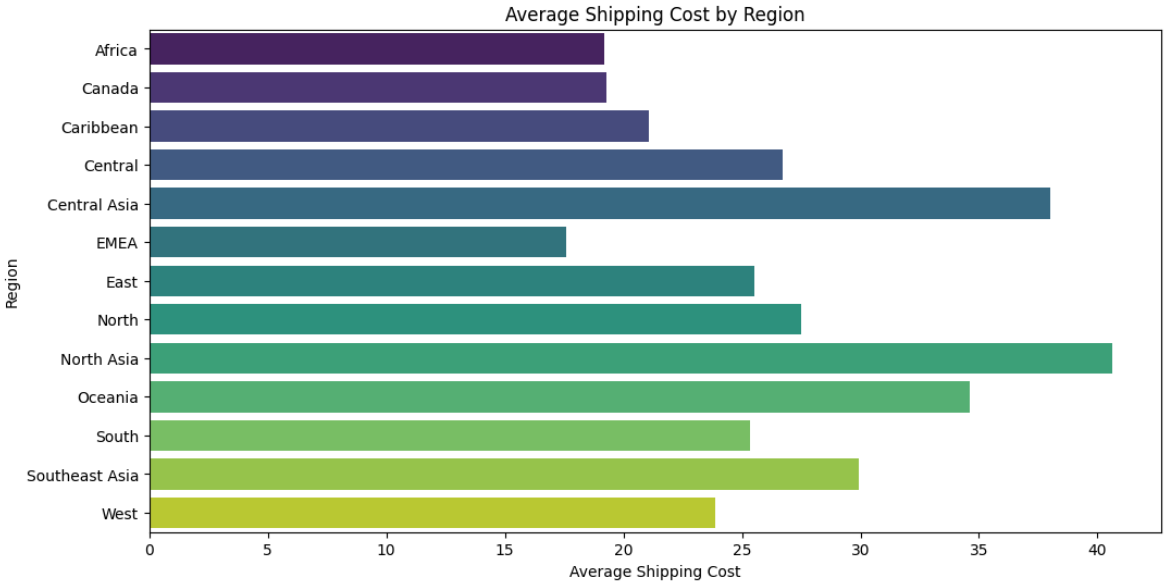
In [ ]:
```
# Plot average shipping cost by region
plt.figure(figsize=(12, 6))
sns.barplot(x='Average Shipping Cost', y='Region', data=region_shipping, palette
plt.title('Average Shipping Cost by Region')
```

```
/tmp/ipykernel_886/137073443.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Average Shipping Cost', y='Region', data=region_shipping, palett
e='viridis')
```

Out[ ]:
```
Text(0.5, 1.0, 'Average Shipping Cost by Region')
```



In [ ]:
```
# Number of customers by city
City_customer_density = data.groupby('City')['Customer ID'].nunique().reset_inde
City_customer_density.columns = ['City',  'Number of Customers']
```

In [ ]:
```
# Number of customers by State
state_customer_density = data.groupby('State')['Customer ID'].nunique().reset_in
state_customer_density.columns = ['State', 'Number of Customers']
```

In [ ]:
```
# Number of customers by Country
country_customer_density = data.groupby('Country')['Customer ID'].nunique().rese
country_customer_density.columns = ['Country', 'Number of Customers']
```

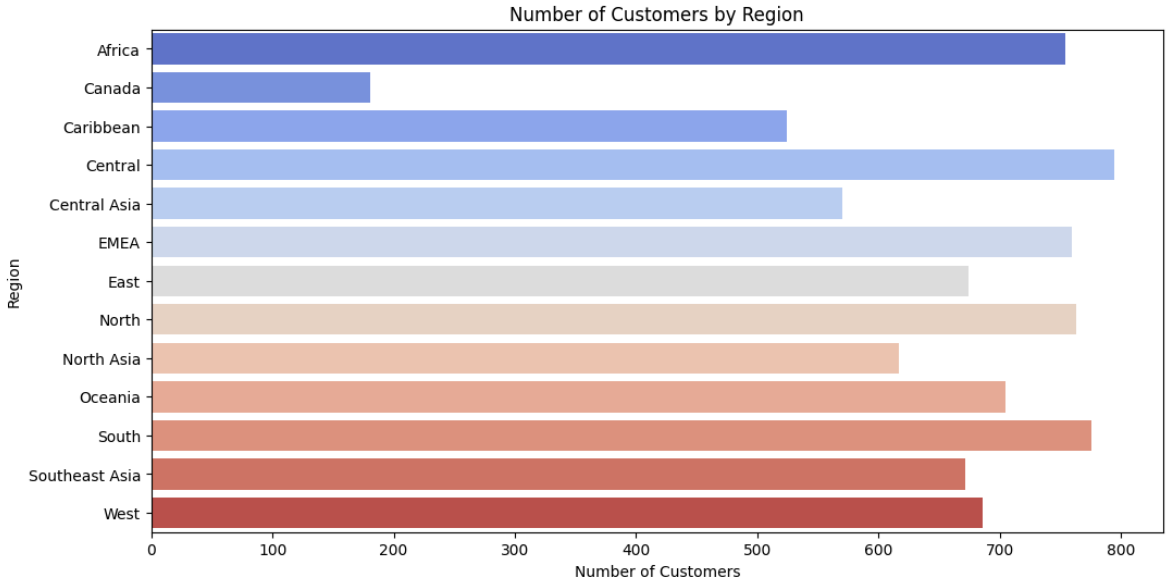
In [ ]:
```
# Number of customers by Region
region_customer_density = data.groupby('Region')['Customer ID'].nunique().reset_
region_customer_density.columns = ['Region', 'Number of Customers']
```

In [ ]:
```
# Plot number of customers by Region
plt.figure(figsize=(12, 6))
sns.barplot(x='Number of Customers', y='Region', data=region_customer_density, p
plt.title('Number of Customers by Region')
plt.xlabel('Number of Customers')
plt.ylabel('Region')
plt.show()
```

```
/tmp/ipykernel_886/3253066131.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Number of Customers', y='Region', data=region_customer_density,
palette='coolwarm')
```



In [ ]:
```
City_customer_density
```

Out[ ]:

	City	Number of Customers
0	Aachen	10
1	Aalen	1
2	Aalst	1
3	Aba	9
4	Abadan	6
...	...	...
3631	Zwedru	1
3632	Zwickau	3
3633	Zwolle	2
3634	eMbalenhle	1
3635	Águas Lindas de Goiás	3

3636 rows × 2 columns